

I. Supported implementations

All implementations are located in the `products` folder. Each implementation provides the following features.

- Modular palette formats (one file per palette).
- Palette definitions in both high-fidelity (original size) and small (currently 40 colors at most) sizes.

Most implementations also feature the API explained below.

- Select specific colors from an existing palette using their indices.
- Shift the palette left (negative value) or right (positive value) by any number of steps.
- Reverse the order of the colors.

Important.

To explain how new palettes can be built, we will refer to the colors in the standard palette as `coul_1`, `coul_2`, etc., and suppose that the extracted indices are `{1, 3, 6, 9}`, the shift used is `1+`, and the `reverse` option is enabled. The new palette will then be built sequentially as follows: first `{coul_1, coul_3, coul_6, coul_9}` (extraction), second `{coul_9, coul_1, coul_3, coul_6}` (shift to the right), and finally `{coul_6, coul_3, coul_1, coul_9}` (reverse).

Note.

Extra features are limited to discrete palette operations. For example, color interpolation is not provided, as this is usually handled out of the box by visualization and formatting tools.

1. JSON, the versatile default format

The JSON product enables seamless `@prism` palette integration for unsupported programming languages. As shown below, palettes are defined using nested arrays of three-component floats.

```
[  
  [0.4980392157, 0.7882352941, 0.4980392157],  
  [0.7450980392, 0.6823529412, 0.831372549],  
  ...  
]
```

Note.

The original and size-40 palettes are located in the `palettes-hf` and `palettes-s40` folders, respectively.